Reconfigurable Test Fixture (RTF) YAML File Documentation

Overview

Reconfigurable Test Fixture (RTF) test files are **UTF-8 encoded YAML files** used to:

- Configure a chip's pin layout
- Define how the test fixture interacts with the chip
- Specify test cases and expected outputs

These files can be:

- Generated automatically via the GUI
- Manually written or edited in any text editor

General Rules

- YAML syntax must be followed (indentation matters)
- All keys are **case-sensitive**
- `#` denotes a comment (everything after it is ignored)
- Only fields defined in this document are supported

File Structure

A valid RTF YAML file typically contains:

```
Chip Info:
Pin Config:
Test1:
Test2:
...
```

---

1. Chip Info (Metadata)

This section provides human-readable information about the chip.

```
Chip Info:
Chip Number: 244
Logic Type: N/A
Number of Inputs: 3
Description: Octal buffers and line drivers with 3-state outputs
Pin Count: 20
```

Notes

- This section is primarily for **documentation and readability**
- It may be used by the GUI or logging system
- Not all fields are strictly required for execution, but recommended

---

2. Pin Configuration

Defines how each physical pin behaves.

Basic Format

```
Pin Config:
1: I
2: O
3: G
4: V
```

Supported Pin Types

Symbol	Meaning
I	Input
O	Output
C	Clock
G	Ground
V	VCC (Power)
C_F	Falling-edge clock
C_R	Rising-edge clock

Expanded Pin Definition (Recommended)

```
Pin Config:
1:
description: Channel 1 output enable
config: I
name: OE
```

Fields

- `config` → Required (pin type)
- `name` → Optional alias for grouping
- `description` → Optional human-readable info

Pin Naming and Grouping

Pins can share the same `name` to allow grouped operations.

```
2:
config: I
name: A
4:
config: I
name: A
```

This allows:

```
A: 1
```

to apply to **both pins simultaneously**.

⚠ **Important:**

Do NOT group pins of different types (e.g., input + output). This will cause errors.

---

3. Writing Tests

Tests are defined as sequential blocks:

```
Test1:
Test2:
Test3:
```

They execute in numeric order.

Basic Test Format

```
Test1:
1: 1
2: 0
3: 1
```

Behavior

- Inputs → values are **driven to the chip**
- Outputs → values are **expected results**

Using Named Pins

```
Test1:
A: 1
Y: 0
```

Applies values to all pins grouped under `A` and `Y`.

Test Descriptions

Tests may include a `description` field:

```
Test1:
description: OE = 0, A = 1, Y = 1
OE: 0
A: 1
Y: 1
```

---

4. Combined Pin Assignments

Multiple pins can be assigned in one line.

Binary Format (Recommended)

```
Test1:
1,2,3,4: 0b1101
```

- `0b` prefix is required for binary interpretation
- Bits map **left to right**

Decimal Format

```
Test1:
1,2,3,4: 13
```

Equivalent to `0b1101`

Using Named Groups

```
Test1:
A,B,Y: 0b011
```

Invalid Example

```
1,CLK,3: 0b1C0 # □ Not allowed
```

- Special commands (like `C`) cannot be used in combined assignments

---

5. Clocking (C Command)

Clock pins can be triggered using:

```
CLK: C
```

- Behavior depends on pin configuration:
  - `C_R` → Rising edge
  - `C_F` → Falling edge

---

6. Sequential Logic (S and T Commands)

Used for flip-flops and stateful circuits.

S Command (Stay the Same)

```
Q: S
```

Behavior:

1. Read current value
2. Apply inputs
3. Read again
4. Verify value **did not change**

T Command (Toggle)

```
Q: T
```

Behavior:

- Output must be the **inverse** of previous value

Example (Flip-Flop Behavior)

```
Test2:
CLK: C
Q: S

Test3:
CLK: C
Q: T
```

7. Example: Sequential Chip (74LS73)

```
Pin Config:
1:
config: C_F
name: CLK
9:
config: O
name: Q
8:
config: O
name: Qnot

Test3:
CLR: 1
J: 1
K: 1
CLK: C
Q: T
Qnot: T
```

---

8. Example: Combinational Chip (74LS244)

```
Test1:
description: OE = 0, A = 1, Y = 1
OE: 0
A: 1
Y: 1
```

Best Practices

- Use `description` fields for clarity
- Prefer named pins over raw numbers
- Keep consistent formatting and indentation
- Use grouping to reduce repetition
- Avoid mixing pin types in groups
- Use binary (`0b`) for combined tests to avoid ambiguity

Summary

RTF YAML files allow you to:

- Define **hardware configuration**
- Write **concise and readable tests**
- Handle both **combinational and sequential logic**

By combining:

- Pin naming
- Grouping
- Binary assignments
- Sequential commands (`S`, `T`, `C`)

you can create powerful and compact test definitions.